

Advanced C++ STL

Interview Questions & Answers

100 Expert-Level Questions with Code Examples

Covering C++11 through C++23

SECTION 1: STL Fundamentals & Architecture

Q1. What are the four main components of the STL?

► Answer:

The STL is composed of: (1) Containers — data structures like vector, list, map; (2) Iterators — objects that generalize pointers to traverse containers; (3) Algorithms — generic functions like sort(), find(), transform(); (4) Function Objects (Functors) — objects callable like functions, enabling customization of algorithms.

Example:

```
#include <vector>
#include <algorithm>
#include <functional>

std::vector<int> v = {5, 2, 8, 1, 9};
// Algorithm + functor (greater<int>) + iterator range
std::sort(v.begin(), v.end(), std::greater<int>());
// v is now {9, 8, 5, 2, 1}
```

Q2. Explain iterator categories and their capabilities.

► Answer:

C++ defines five iterator categories: Input (read-only, single-pass), Output (write-only, single-pass), Forward (read/write, multi-pass, moves forward), Bidirectional (like Forward + moves backward), and Random Access (all above + O(1) arbitrary jumps). Stronger categories are backward compatible with weaker ones.

Example:

```
// Random Access: std::vector, std::deque
std::vector<int> v = {1,2,3,4,5};
auto it = v.begin();
it += 3; // Jump ahead 3 – only RA iterators!
std::cout << *it; // 4

// Bidirectional: std::list, std::map
std::list<int> lst = {10,20,30};
auto li = lst.end();
--li; // Move back – bidirectional
std::cout << *li; // 30
```

Q3. What is iterator invalidation and when does it occur?

► Answer:

Iterator invalidation happens when a container operation makes existing iterators undefined. For vector: any insertion that causes reallocation invalidates all iterators; insertions before an iterator invalidate it and all after it; erasure invalidates iterators at and after the erased position. For map/set: iterators are stable across insertions/erasures of other elements.

Example:

```
std::vector<int> v = {1, 2, 3, 4, 5};
auto it = v.begin() + 2; // Points to 3

// DANGEROUS: erase invalidates it and beyond
v.erase(v.begin()); // it is now INVALID!
```

```
// SAFE pattern: capture returned iterator
it = v.erase(v.begin()); // it now points to next valid
std::cout << *it; // 2 (formerly at index 1)
```

Q4. What is the difference between `size()` and `capacity()` in `std::vector`?

▶ Answer:

`size()` returns the number of elements currently stored. `capacity()` returns how many elements the vector can hold before needing to reallocate. The vector pre-allocates memory in chunks (usually doubling) to amortize $O(1)$ `push_back` cost. `capacity` $\geq$ `size` always. You can use `reserve()` to pre-allocate and `shrink_to_fit()` to release excess.

📄 Example:

```
std::vector<int> v;
v.reserve(100);
std::cout << v.size();    // 0
std::cout << v.capacity(); // 100

v.push_back(42);
std::cout << v.size();    // 1
std::cout << v.capacity(); // 100 (no realloc)

v.shrink_to_fit();
std::cout << v.capacity(); // 1 (implementation-defined)
```

Q5. How does `std::vector` grow? What is amortized $O(1)$?

▶ Answer:

When `push_back` exceeds capacity, the vector allocates a new buffer (typically 2x the current capacity), copies all elements, then frees the old buffer. Though each such reallocation is $O(n)$, it happens so rarely that over n insertions the total work is $O(n)$ — making the average (amortized) cost per `push_back` $O(1)$.

📄 Example:

```
std::vector<int> v;
for (int i = 0; i < 16; ++i) {
    v.push_back(i);
    std::cout << "size=" << v.size()
              << " cap=" << v.capacity() << "\n";
}
// Typical output: capacity doubles at 1,2,4,8,16...
```

SECTION 2: Sequence Containers

Q6. When should you use `std::deque` over `std::vector`?

▶ Answer:

Use deque when you need $O(1)$ insertions/deletions at both ends (vector only has $O(1)$ at the back). deque uses a map of fixed-size chunks, so it avoids single large reallocations. However, deque has worse cache

locality than vector and slower random-access constants. Prefer vector unless you specifically need efficient front insertion.

Example:

```
std::deque<int> dq;
dq.push_back(1); // O(1) – back insert
dq.push_front(0); // O(1) – front insert (vector lacks this!)
dq.pop_front(); // O(1) – front removal
std::cout << dq[0]; // O(1) random access
```

Q7. Explain the trade-offs of `std::list` vs `std::vector`.

Answer:

`std::list` is a doubly-linked list with $O(1)$ insertion/deletion anywhere if you have an iterator, and stable iterators under any modification. `std::vector` offers $O(1)$ random access and superior cache performance. In practice, vector almost always wins due to cache effects — even $O(n)$ vector operations can beat $O(1)$ list operations in real benchmarks because list nodes are scattered in memory.

Example:

```
std::list<int> lst = {1, 2, 3, 4, 5};
auto it = std::find(lst.begin(), lst.end(), 3);

// O(1) insert before position – iterator stays valid
lst.insert(it, 99); // {1, 2, 99, 3, 4, 5}

// O(1) erase – returns valid next iterator
it = lst.erase(it); // {1, 2, 99, 4, 5}, it->4
```

Q8. What is `std::array` and when should you use it over C arrays?

Answer:

`std::array<T,N>` is a fixed-size stack-allocated array that wraps a C array with a value-type interface: it supports iterators, range-for, `size()`, `at()` with bounds checking, and works with STL algorithms. Unlike C arrays, it does not decay to a pointer and can be copied/returned. Use it whenever the size is known at compile time.

Example:

```
std::array<int, 5> arr = {3, 1, 4, 1, 5};

// Works with STL algorithms
std::sort(arr.begin(), arr.end());

// Bounds-checked access
try { arr.at(10); }
catch (std::out_of_range& e) {
    std::cout << e.what();
}

// Can be returned by value (unlike C arrays)
auto getCopy = [&]() -> std::array<int,5> { return arr; };
```

Q9. What does `emplace_back()` do differently from `push_back()`?

Answer:

`push_back()` copies or moves an already-constructed object. `emplace_back()` forwards arguments directly to the element constructor, constructing in-place inside the vector's storage — no temporary object created. For complex objects this avoids a copy/move entirely. With trivial types or when passing existing objects the difference is minimal.

Example:

```
struct Point { int x, y; Point(int x, int y): x(x), y(y){} };

std::vector<Point> v;

// push_back: constructs temporary, then moves/copies it in
v.push_back(Point{1, 2});

// emplace_back: forwards args, constructs IN PLACE
v.emplace_back(3, 4); // No temporary Point created
```

Q10. How do `std::span` (C++20) and `std::string_view` improve performance?

Answer:

Both are non-owning views — they hold a pointer and a size without copying data. `string_view` replaces `const string&` parameters, avoiding construction of `std::string` from string literals. `span<T>` is the equivalent for contiguous sequences (array, vector). They eliminate unnecessary allocations in read-only contexts and work with multiple container types.

Example:

```
#include <span>
#include <string_view>

void processData(std::span<const int> data) {
    for (int x : data) std::cout << x << " ";
}

std::vector<int> v = {1,2,3,4,5};
std::array<int,3> a = {6,7,8};
int raw[] = {9,10,11};

processData(v);    // Works with vector
processData(a);    // Works with array
processData(raw);  // Works with C array – no copies!
```

SECTION 3: Associative & Unordered Containers

Q11. What is the underlying data structure of `std::map` and `std::set`?

Answer:

Both use a self-balancing Red-Black Tree. This guarantees $O(\log n)$ for insert, erase, and find. Elements are stored in sorted order. The tree structure means iterating over a map yields elements in key order. Nodes are heap-allocated individually, which gives stable iterators/references but poor cache locality compared to hash maps.

Example:

```
std::map<std::string, int> scores;
```

```
scores["Alice"] = 95;
scores["Bob"]   = 87;
scores["Carol"] = 91;

// Iteration is always in sorted key order
for (auto& [name, score] : scores)
    std::cout << name << ": " << score << "\n";
// Alice:95 Bob:87 Carol:91
```

Q12. What is the complexity and data structure behind `std::unordered_map`?

▶ Answer:

`std::unordered_map` uses a hash table (array of buckets, each a linked list). Average-case $O(1)$ insert/erase/find. Worst-case $O(n)$ when all keys hash to the same bucket. The hash function must be provided for custom key types. It has higher constant factors than `map` for small data sets and uses more memory for the bucket array.

Example:

```
std::unordered_map<std::string, int> freq;

for (std::string word : {"cat", "dog", "cat", "fish", "dog", "cat"})
    freq[word]++;

// O(1) average lookup
std::cout << freq["cat"]; // 3

// Custom hash for user-defined key:
struct PairHash {
    size_t operator()(const std::pair<int, int>& p) const {
        return std::hash<int>()(p.first) ^
               (std::hash<int>()(p.second) << 32);
    }
};
std::unordered_map<std::pair<int, int>, int, PairHash> grid;
```

Q13. Explain the difference between `operator[]` and `at()` in `std::map`.

▶ Answer:

`operator[]` inserts a default-constructed value if the key doesn't exist — it is NOT const-qualified. `at()` throws `std::out_of_range` for missing keys and IS const-qualified, making it safe for const maps. Prefer `at()` for lookups where the key should exist, and `find()` when absence is expected.

Example:

```
std::map<std::string, int> m;
m["a"] = 1;

// operator[]: creates entry with value 0!
std::cout << m["b"]; // 0 — "b" now exists in map
std::cout << m.size(); // 2

// at(): safe lookup, throws if missing
try { m.at("c"); }
catch(std::out_of_range&) { std::cout << "not found"; }

// find(): check before access
if (auto it = m.find("a"); it != m.end())
    std::cout << it->second; // 1
```

Q14. What are `std::multimap` and `std::multiset`? How do you iterate over equal keys?**▶ Answer:**

`multimap` and `multiset` allow duplicate keys. They still maintain sorted order. To access all values with a given key, use `equal_range()` which returns a pair of iterators [first, last) over all matching elements. `count()` returns how many elements match. `insert()` always succeeds (unlike `map` where it fails for existing keys).

Example:

```
std::multimap<std::string, int> mm;
mm.insert({"Alice", 90});
mm.insert({"Alice", 85});
mm.insert({"Alice", 92});
mm.insert({"Bob", 78});

auto [lo, hi] = mm.equal_range("Alice");
for (auto it = lo; it != hi; ++it)
    std::cout << it->second << " "; // 90 85 92

std::cout << mm.count("Alice"); // 3
```

Q15. How do you provide a custom comparator to `std::set`?**▶ Answer:**

`std::set<Key, Compare>` accepts a comparator type as a template parameter. The comparator must define a strict weak ordering (irreflexive, asymmetric, transitive). You can use a functor class, a lambda with `decltype`, or `std::function`. Lambdas are easiest in C++20 with the comparator passed to the constructor.

Example:

```
// Using a functor (pre-C++20)
struct CaseInsensitive {
    bool operator()(const std::string& a,
                    const std::string& b) const {
        return std::lexicographical_compare(
            a.begin(), a.end(), b.begin(), b.end(),
            [](char x, char y){ return tolower(x)<tolower(y); });
    }
};

std::set<std::string, CaseInsensitive> s;
s.insert("Banana"); s.insert("apple"); s.insert("Cherry");
// Ordered: apple, Banana, Cherry
```

Q16. What is a load factor and rehashing in unordered containers?**▶ Answer:**

Load factor = number of elements / number of buckets. When `load_factor()` exceeds `max_load_factor()` (default 1.0), the container rehashes — allocates more buckets and redistributes all elements. This is $O(n)$ but rare. Use `reserve(n)` to pre-size the bucket array and avoid rehashes when inserting n elements.

Example:

```
std::unordered_map<int, int> um;
um.reserve(1000); // Pre-allocate for 1000 elements
um.max_load_factor(0.5); // Rehash at 50% full (fewer collisions)

std::cout << um.bucket_count(); // e.g. 2048
```

```
for (int i = 0; i < 800; ++i) um[i] = i;
std::cout << um.load_factor(); // ~0.39 (no rehash needed)
```

SECTION 4: Container Adaptors

Q17. What is `std::stack` and what underlying container does it use?

► Answer:

`std::stack` is a LIFO adaptor defaulting to `std::deque` as its underlying container, but you can use `vector` or `list`. It exposes only `top()`, `push()`, `pop()`, `empty()`, `size()`. No iterators — you cannot traverse a stack. Using `vector` as the underlying container is often faster due to better cache locality.

Example:

```
std::stack<int> s; // Uses deque by default
std::stack<int, std::vector<int>> sv; // Use vector instead

sv.push(1); sv.push(2); sv.push(3);
std::cout << sv.top(); // 3
sv.pop();
std::cout << sv.top(); // 2
```

Q18. How does `std::priority_queue` work and what is its complexity?

► Answer:

`std::priority_queue` is a max-heap adaptor over a random-access container (default: `vector`). `push()` inserts and bubbles up: $O(\log n)$. `top()` returns max element: $O(1)$. `pop()` removes max and restores heap property: $O(\log n)$. For a min-heap, use `greater<T>` as comparator. Heap construction from a range is $O(n)$.

Example:

```
// Max-heap (default)
std::priority_queue<int> maxHeap;
for (int x : {3,1,4,1,5,9}) maxHeap.push(x);
std::cout << maxHeap.top(); // 9

// Min-heap
using MinHeap = std::priority_queue<int,
    std::vector<int>, std::greater<int>>>;
MinHeap minH;
for (int x : {3,1,4,1,5,9}) minH.push(x);
std::cout << minH.top(); // 1
```

Q19. When would you use `std::queue` vs `std::deque` directly?

► Answer:

Use `std::queue` when you want a restricted FIFO interface (`push/pop/front/back` only) to prevent accidental random access. `std::deque` gives full deque functionality including iterators and random access. `queue` is a design tool — it communicates intent and prevents misuse. Internally, `queue` defaults to `deque`.

 Example:

```
std::queue<std::string> bfsQueue;
bfsQueue.push("start");

while (!bfsQueue.empty()) {
    auto node = bfsQueue.front();
    bfsQueue.pop();
    // Process node...
    // bfsQueue.push(neighbor); – typical BFS usage
}
```

SECTION 5: STL Algorithms

Q20. What is the difference between `std::sort`, `std::stable_sort`, and `std::partial_sort`?

 Answer:

`std::sort` is $O(n \log n)$ average, not stable (equal elements may reorder) — uses introsort. `std::stable_sort` is $O(n \log^2 n)$ or $O(n \log n)$ with extra memory, preserves relative order of equal elements. `std::partial_sort`(first, middle, last) puts the smallest (middle-first) elements in sorted order in $[first, middle)$ in $O(n \log k)$ where $k = middle - first$.

 Example:

```
std::vector<int> v = {5, 2, 8, 1, 9, 3, 7, 4, 6};

// Sort entire range
std::sort(v.begin(), v.end()); //  $O(n \log n)$ 

// Stable sort preserves relative order of equals
std::stable_sort(v.begin(), v.end());

// Get the 3 smallest elements sorted
std::partial_sort(v.begin(), v.begin()+3, v.end());
// First 3 positions: {1, 2, 3, ...rest unsorted}
```

Q21. Explain `std::lower_bound` and `std::upper_bound`.

 Answer:

Both perform binary search on sorted ranges: $O(\log n)$. `lower_bound` returns an iterator to the first element NOT LESS THAN value. `upper_bound` returns an iterator to the first element GREATER THAN value. Together they define the half-open range $[lower, upper)$ of elements equal to value. They also work on sorted `std::map` and `std::set` as member functions.

 Example:

```
std::vector<int> v = {1, 2, 2, 3, 3, 3, 4, 5};

auto lo = std::lower_bound(v.begin(), v.end(), 3);
auto hi = std::upper_bound(v.begin(), v.end(), 3);

std::cout << *lo; // 3 (first 3)
std::cout << std::distance(lo, hi); // 3 (count of 3s)
```

```
// Check membership
bool found = (lo != v.end() && *lo == 3); // true
```

Q22. What does `std::transform` do and how is it used?

► Answer:

`std::transform` applies a function to each element of a range and stores the result in an output range. It has a unary form (one input range) and a binary form (two input ranges). The output range can be the same as the input for in-place transformation, or another container.

📄 Example:

```
std::vector<int> v = {1, 2, 3, 4, 5};
std::vector<int> out(v.size());

// Unary: square each element
std::transform(v.begin(), v.end(), out.begin(),
    [](int x){ return x * x; });
// out: {1, 4, 9, 16, 25}

// Binary: add two ranges element-wise
std::vector<int> a={1,2,3}, b={10,20,30}, c(3);
std::transform(a.begin(), a.end(), b.begin(), c.begin(),
    std::plus<int>());
// c: {11, 22, 33}
```

Q23. What is `std::accumulate` and how does it differ from `std::reduce`?

► Answer:

`std::accumulate` applies a binary operation left-to-right in order, starting with an init value — it is inherently sequential and deterministic. `std::reduce` (C++17) can execute in any order, enabling parallel execution with execution policies. `reduce` requires the operation to be associative and commutative for correct results.

📄 Example:

```
#include <numeric>
#include <execution>

std::vector<int> v = {1,2,3,4,5};

// accumulate: sequential, ordered
int sum = std::accumulate(v.begin(), v.end(), 0);
// Product with custom op:
int prod = std::accumulate(v.begin(), v.end(), 1,
    std::multiplies<int>());

// reduce: parallel-friendly
int psum = std::reduce(std::execution::par,
    v.begin(), v.end(), 0);
```

Q24. Explain `std::copy_if`, `std::remove_if`, and the erase-remove idiom.

► Answer:

`std::remove_if` does NOT erase elements — it moves "kept" elements to the front and returns an iterator to the new logical end; the elements beyond it are in a valid but unspecified state. To actually erase them you must pair with `container.erase()`. `copy_if` writes elements satisfying a predicate to an output range.

 Example:

```
std::vector<int> v = {1,2,3,4,5,6,7,8,9,10};

// Erase-remove idiom: remove even numbers
v.erase(std::remove_if(v.begin(), v.end(),
    [](int x){ return x % 2 == 0; }),
    v.end());
// v: {1,3,5,7,9}

// copy_if to a new container
std::vector<int> evens;
std::copy_if(v.begin(), v.end(),
    std::back_inserter(evens),
    [](int x){ return x % 2 == 0; });
```

Q25. What are execution policies (C++17) and how do they enable parallelism? **Answer:**

C++17 adds three execution policies: `std::execution::seq` (sequential), `std::execution::par` (parallel), `std::execution::par_unseq` (parallel + vectorized). Pass as first argument to most algorithms to hint the runtime to use thread pools or SIMD. Predicates/functions must be thread-safe with `par`. Not all implementations support parallel execution.

 Example:

```
#include <execution>
#include <algorithm>

std::vector<int> v(1000000);
std::iota(v.begin(), v.end(), 0);

// Sequential (default)
std::sort(std::execution::seq, v.begin(), v.end());

// Parallel
std::sort(std::execution::par, v.begin(), v.end());

// Parallel + SIMD
std::for_each(std::execution::par_unseq,
    v.begin(), v.end(), [](int& x){ x *= 2; });
```

Q26. What is `std::nth_element` and what guarantee does it provide? **Answer:**

`std::nth_element` rearranges a range so that the element at the `nth` position is the element that would be there if the range were sorted, all elements before it are $\leq$ it, and all after are $\geq$ it. It runs in $O(n)$ average (introsselect). It is ideal for finding medians or top-k elements without full sorting.

 Example:

```
std::vector<int> v = {5,2,8,1,9,3,7,4,6};

auto mid = v.begin() + v.size()/2;
std::nth_element(v.begin(), mid, v.end());

// *mid is the median value
std::cout << *mid; // 5
// Elements before mid are <= 5
// Elements after mid are >= 5
```

```
// (but not sorted within those groups)
```

Q27. How do `std::find`, `std::find_if`, and `std::find_if_not` differ?

▶ Answer:

`std::find` searches for a specific value by equality. `std::find_if` searches for the first element satisfying a unary predicate. `std::find_if_not` finds the first element NOT satisfying the predicate. All return `end()` if nothing is found and run in $O(n)$ on unsorted ranges.

▒ Example:

```
std::vector<int> v = {1,3,5,7,8,10,11};

// Find first even number
auto it = std::find_if(v.begin(), v.end(),
    [](int x){ return x % 2 == 0; });
std::cout << *it; // 8

// Find first non-odd (same result here)
auto it2 = std::find_if_not(v.begin(), v.end(),
    [](int x){ return x % 2 != 0; });
std::cout << *it2; // 8
```

Q28. Explain `std::for_each` vs range-based for loop.

▶ Answer:

Range-based for is simpler syntax for iterating and modifying via reference. `std::for_each` takes an iterator range (useful for sub-ranges), returns the function object (useful to retrieve state from the functor), and works with execution policies for parallelism. Prefer range-for for full-container iteration; prefer `for_each` for sub-ranges or when you need the function object returned.

▒ Example:

```
std::vector<int> v = {1,2,3,4,5};

// Range-for (simpler for full range)
for (int& x : v) x *= 2;

// for_each on sub-range
std::for_each(v.begin()+1, v.end()-1,
    [](int& x){ x += 10; });
// Only modifies elements at index 1,2,3

// for_each returns the functor (useful for stateful lambdas)
int sum = 0;
std::for_each(v.begin(), v.end(),
    [&sum](int x){ sum += x; });
```

SECTION 6: Iterators & Ranges

Q29. What are iterator adaptors and give examples.**▶ Answer:**

Iterator adaptors wrap iterators to provide new behavior. Key examples: `std::reverse_iterator` reverses traversal direction; `std::back_insert_iterator` calls `push_back` on assignment; `std::front_insert_iterator` calls `push_front`; `std::insert_iterator` calls `insert`; `std::move_iterator` dereferences to rvalue references (moves instead of copies).

Example:

```
std::vector<int> src = {1,2,3,4,5};
std::vector<int> dst;

// back_inserter: auto push_back on write
std::copy(src.begin(), src.end(),
          std::back_inserter(dst));

// reverse_iterator: traverse backwards
for (auto it = src.rbegin(); it != src.rend(); ++it)
    std::cout << *it << " "; // 5 4 3 2 1

// move_iterator: move elements during algorithm
std::vector<std::string> s1={"a","b","c"}, s2;
std::copy(std::make_move_iterator(s1.begin()),
          std::make_move_iterator(s1.end()),
          std::back_inserter(s2));
```

Q30. What are C++20 Ranges and how do they improve on iterator pairs?**▶ Answer:**

C++20 Ranges provide lazy, composable views over sequences. Instead of passing begin/end pairs, you pass a range and can chain views with `|` (pipe). Ranges are lazy — no intermediate allocations. `std::views::filter`, `transform`, `take`, `drop`, `reverse`, etc. are standard views. They enable concise, readable data pipelines with zero overhead.

Example:

```
#include <ranges>

std::vector<int> v = {1,2,3,4,5,6,7,8,9,10};

// Chain views lazily
auto result = v
    | std::views::filter([](int x){ return x%2==0; })
    | std::views::transform([](int x){ return x*x; })
    | std::views::take(3);

for (int x : result)
    std::cout << x << " "; // 4 16 36
// No intermediate containers allocated!
```

Q31. What is `std::istream_iterator` and how is it used?**▶ Answer:**

`std::istream_iterator<T>` treats an `istream` as an input range. When dereferenced, it reads the next `T` from the stream. The default-constructed `istream_iterator` is the end-of-stream sentinel. This allows you to use stream input with STL algorithms like `copy` and for range-based construction of containers.

Example:

```
#include <iterator>
#include <sstream>

std::istringstream iss("10 20 30 40 50");

// Construct vector from stream
std::vector<int> v{
    std::istream_iterator<int>(iss),
    std::istream_iterator<int>()};
// v: {10,20,30,40,50}

// Sum from stdin
// int sum = std::accumulate(
//     std::istream_iterator<int>(std::cin),
//     std::istream_iterator<int>(), 0);
```

SECTION 7: Function Objects & Lambdas

Q32. What is a functor and why is it preferred over function pointers in STL?

► Answer:

A functor (function object) is a class with operator() defined. Unlike function pointers, functors can carry state (member variables), are inlineable by the compiler (no indirection), and support template specialization. The STL's algorithm performance depends on the optimizer being able to inline the comparison/predicate — functors make this possible.

Example:

```
struct Multiplier {
    int factor;
    Multiplier(int f) : factor(f) {}
    int operator()(int x) const { return x * factor; }
};

std::vector<int> v = {1,2,3,4,5};
std::transform(v.begin(), v.end(), v.begin(),
    Multiplier(3));
// v: {3,6,9,12,15}

// Lambda is syntactic sugar for an anonymous functor
std::transform(v.begin(), v.end(), v.begin(),
    [factor=3](int x){ return x * factor; });
```

Q33. Explain lambda capture modes: [=], [&], [this], explicit captures.

► Answer:

[=] captures all local variables by value (copy). [&] captures all by reference. [this] captures the current object pointer. Explicit captures like [x, &y] capture x by value and y by reference. In C++14, init captures [z=expr] allow naming captures. Prefer explicit captures for clarity and to avoid dangling references.

Example:

```
int a = 10, b = 20;
```

```
// By value – copies at capture time
auto f1 = [=]() { return a + b; };
a = 99;
std::cout << f1(); // 30 (captured old a=10)

// By reference – reads current value
auto f2 = [&]() { return a + b; };
std::cout << f2(); // 119 (reads a=99)

// Mixed capture
auto f3 = [a, &b]() { return a + b; };

// Init capture (C++14)
auto f4 = [val = a*2]() { return val; }; // val=198
```

Q34. What are `std::function` and `std::bind`? When should you avoid them?

▶ Answer:

`std::function` is a type-erased callable wrapper — it can hold lambdas, function pointers, or functors with the same signature. `std::bind` partially applies arguments. Both incur overhead: `std::function` has potential heap allocation and virtual dispatch; `std::bind` has complex type inference. For performance-critical code prefer templates/lambdas over `std::function`.

📄 Example:

```
std::function<int(int,int)> add = [](int a, int b){ return a+b; };
std::cout << add(3, 4); // 7

// std::bind: partial application
auto add5 = std::bind(add, std::placeholders::_1, 5);
std::cout << add5(3); // 8

// PREFER lambdas over bind (clearer, faster):
auto add5_lambda = [&add](int x){ return add(x, 5); };
std::cout << add5_lambda(3); // 8
```

Q35. What are the standard function objects in `<functional>`?

▶ Answer:

The `<functional>` header provides arithmetic operators (plus, minus, multiplies, divides, modulus, negate), comparators (equal_to, not_equal_to, less, greater, less_equal, greater_equal), logical operators (logical_and, logical_or, logical_not), and bitwise operators (bit_and, bit_or, bit_xor, bit_not). They are transparent when parameterized with void (e.g. `std::less<void>`).

📄 Example:

```
// Use standard functors instead of lambdas for common ops
std::vector<int> v = {3,1,4,1,5,9,2,6};

// Sort descending
std::sort(v.begin(), v.end(), std::greater<int>());

// Sum with plus<>
int total = std::accumulate(v.begin(), v.end(), 0,
    std::plus<int>());

// Transparent comparator: std::less<void>
// Can compare different types without casting
std::set<int, std::less<>> s = {1,2,3,4,5};
```

```
std::cout << (s.find(3) != s.end()); // 1
```

SECTION 8: Memory Management in STL

Q36. What is an allocator in STL and why would you write a custom one?

► Answer:

An allocator is a policy class that controls how STL containers acquire and release memory. The default `std::allocator` uses global `new/delete`. Custom allocators enable: pool allocation for performance, arena/monotonic allocation for better cache locality, tracking/debugging allocations, or using shared memory. C++17 added `std::pmr` (polymorphic memory resources) as a simpler alternative.

Example:

```
#include <memory_resource>

// PMR: stack-based monotonic buffer (no heap!)
std::byte buffer[4096];
std::pmr::monotonic_buffer_resource pool{buffer, sizeof(buffer)};

std::pmr::vector<int> v{&pool};
for (int i = 0; i < 100; ++i)
    v.push_back(i); // Allocates from buffer, not heap

// Great for short-lived, high-frequency allocations
```

Q37. Explain `std::unique_ptr`, `std::shared_ptr`, and `std::weak_ptr`.

► Answer:

`unique_ptr` has sole ownership — non-copyable, movable, zero overhead over raw pointer. `shared_ptr` uses reference counting (thread-safe) for shared ownership. `weak_ptr` holds a non-owning reference to `shared_ptr`'s object, preventing the reference count from keeping the object alive — used to break cycles. Prefer `unique_ptr` by default.

Example:

```
auto up = std::make_unique<int>(42);
// auto up2 = up; // ERROR: not copyable
auto up2 = std::move(up); // up is now null

auto sp1 = std::make_shared<std::string>("hello");
auto sp2 = sp1; // Both own the string
std::cout << sp1.use_count(); // 2

// Weak pointer to break cycles
std::weak_ptr<std::string> wp = sp1;
if (auto locked = wp.lock()) // Convert to shared_ptr
    std::cout << *locked; // "hello"
```

Q38. What is `std::move` and move semantics in the context of STL?

Answer:

`std::move` casts an lvalue to an rvalue reference, enabling move semantics — transferring resources rather than copying. STL containers implement move constructors/assignment. Moving a vector is $O(1)$ (pointer swap) vs $O(n)$ copy. `push_back/emplace_back` use move semantics automatically for temporaries. `move` is critical for performance with large or non-copyable objects.

Example:

```
std::vector<std::string> names = {"Alice", "Bob", "Carol"};

// Move vector: O(1), just pointer swap
std::vector<std::string> copy = names;    // O(n) copy
std::vector<std::string> moved = std::move(names); // O(1)!
// names is now in valid but unspecified state (empty)

// Move into container (avoids copy)
std::string s = "expensive string";
copy.push_back(std::move(s)); // s becomes empty
```

SECTION 9: Advanced Containers & C++17/20 Features

Q39. What is `std::optional` and how does it relate to STL usage?

Answer:

`std::optional<T>` represents a value that may or may not be present, without using pointers or sentinel values. It is value-semantic, stack-allocated, and eliminates null pointer issues. In STL patterns, it is ideal for functions that might not find a result — better than returning a `pair<bool,T>` or checking for `end()`.

Example:

```
#include <optional>

std::optional<int> findFirst(const std::vector<int>& v, int target) {
    auto it = std::find(v.begin(), v.end(), target);
    if (it != v.end()) return *it;
    return std::nullopt;
}

auto result = findFirst({1,2,3,4,5}, 3);
if (result) std::cout << *result; // 3
std::cout << result.value_or(-1); // 3

auto missing = findFirst({1,2,3}, 99);
std::cout << missing.value_or(-1); // -1
```

Q40. What is `std::variant` and how is it different from union?

Answer:

`std::variant<Types...>` is a type-safe discriminated union — it knows which type it currently holds and prevents accessing the wrong type. Unlike union, it runs constructors/destructors correctly. Access via `std::get<T>()` (throws if wrong type), `std::get_if<T>()` (returns pointer, null if wrong), or `std::visit()` with a visitor (callable for each possible type).

Example:

```
#include <variant>

using Value = std::variant<int, double, std::string>;

Value v = 42;
std::cout << std::get<int>(v); // 42

v = "hello";
std::visit([](auto&& val){
    std::cout << val; // Handles any type
}, v);

// Safe access
if (auto* p = std::get_if<int>(&v))
    std::cout << *p;
else
    std::cout << "not an int";
```

Q41. Explain `std::any` and when to use it.

▶ Answer:

`std::any` holds a value of any copy-constructible type, type-erasing it. Unlike `variant`, the set of types is open (not fixed at compile time). Useful for type-erased plugin/extension systems, message-passing where content type varies. Overhead is higher than `variant` (potential heap allocation for large types). Access via `std::any_cast<T>()`.

Example:

```
#include <any>

std::any a = 42;
std::cout << std::any_cast<int>(a); // 42

a = std::string("hello");
std::cout << std::any_cast<std::string>(a); // hello

try {
    std::any_cast<int>(a); // Throws bad_any_cast
} catch (std::bad_any_cast& e) {
    std::cout << e.what();
}

std::cout << a.type().name(); // std::string
```

Q42. What are `flat_map` and `flat_set` (C++23) and why are they useful?

▶ Answer:

`std::flat_map` and `std::flat_set` (C++23) are sorted associative containers backed by contiguous arrays (vectors) rather than trees. They provide superior cache performance for lookup-heavy workloads, smaller memory footprint, and faster iteration. Trade-off: $O(n)$ insert/erase. Ideal when the container is built once then queried many times.

Example:

```
// C++23
#include <flat_map>

std::flat_map<std::string, int> fm;
fm.insert({"Alice", 90});
```

```

fm.insert({"Bob", 85});
// Internally: sorted contiguous array
// Lookup: O(log n) binary search on cache-friendly data
// Insert: O(n) shift – use when mostly read

// Pre-C++23 equivalent pattern:
std::vector<std::pair<std::string,int>> sorted_v;
sorted_v.push_back({"Alice",90});
sorted_v.push_back({"Bob",85});
std::sort(sorted_v.begin(), sorted_v.end());
// then use lower_bound for O(log n) search

```

SECTION 10: Performance, Pitfalls & Best Practices

Q43. What is the Small Buffer Optimization (SBO) in `std::string`?

▶ Answer:

Most `std::string` implementations store short strings (typically < 15-23 chars) inside the string object itself, avoiding heap allocation entirely. The object union-stores either a pointer to heap memory or an inline char array. This makes short string construction essentially free and improves cache locality. You can verify by checking if `&s[0]` is inside the object's address range.

📄 Example:

```

std::string short_s = "Hello"; // No heap alloc (SBO)
std::string long_s(100, 'x'); // Heap allocated

// Detecting SBO (implementation-specific)
std::string s = "Hi";
const char* data = s.data();
const char* obj = reinterpret_cast<const char*>(&s);
bool is_sbo = (data >= obj &&
               data < obj + sizeof(s));
std::cout << (is_sbo ? "SBO" : "heap"); // SBO

```

Q44. What is the most common mistake when using `std::map::operator[]`?

▶ Answer:

`operator[]` default-constructs a value for missing keys, silently inserting them. This is dangerous in read-only contexts (breaks const-correctness since it is not const-qualified), inflates map size unintentionally, and can mask bugs where a key is expected to exist. Use `find()` or `at()` instead for safe lookups.

📄 Example:

```

std::map<std::string, std::vector<int>> m;
m["a"] = {1, 2, 3};

// BUG: creates empty vector for "b"!
if (m["b"].empty()) { /* "b" now in map */ }
std::cout << m.size(); // 2, not 1!

// CORRECT: use find()
auto it = m.find("c");
if (it != m.end() && !it->second.empty()) {

```

```
// safe
}
```

Q45. How do you efficiently insert a range of elements into a container?

Answer:

Always prefer range insert (`insert(end, first, last)` or `assign()`) over repeated single inserts for vector. For unordered containers, call `reserve()` first. For map/set, providing a hint iterator gives amortized $O(1)$ if the hint is correct. Constructing with a range directly is fastest for initialization.

Example:

```
std::vector<int> v;
std::vector<int> src = {1,2,3,4,5};

// SLOW: repeated push_back may cause multiple reallocations
for (int x : src) v.push_back(x);

// FAST: single reallocation
v.reserve(src.size());
v.insert(v.end(), src.begin(), src.end());

// FASTEST: construct directly
std::vector<int> v2(src.begin(), src.end());

// Map hint insert ( $O(1)$  amortized if correct)
std::map<int,int> m;
auto hint = m.end();
for (int i = 0; i < 1000; ++i)
    hint = m.insert(hint, {i, i*i});
```

Q46. What is structured bindings (C++17) and how do they work with STL?

Answer:

Structured bindings (`auto [a,b] = expr`) decompose a struct, pair, tuple, or array into named variables. They dramatically improve readability with STL pair-returning functions (`equal_range`, `insert`, `emplace`), map iteration, and tuple usage.

Example:

```
std::map<std::string, int> m = {"a",1},{"b",2};

// Old style
for (auto& kv : m)
    std::cout << kv.first << "=" << kv.second;

// C++17 structured bindings
for (auto& [key, value] : m)
    std::cout << key << "=" << value;

// With insert result
auto [it, inserted] = m.insert({"c", 3});
std::cout << inserted; // true

// With equal_range
auto [lo, hi] = m.equal_range("b");
```

Q47. What is the difference between `std::copy` and `std::move` algorithm?**Answer:**

`std::copy` copies elements from source to destination using copy assignment. `std::move` (the algorithm, not the cast) moves elements using move assignment, leaving the source in a valid but unspecified state. Use `std::move` algorithm when you want to transfer ownership, such as moving elements from one container to another.

Example:

```
std::vector<std::string> src = {"a", "b", "c"};
std::vector<std::string> dst(3);

// copy: src unchanged
std::copy(src.begin(), src.end(), dst.begin());

// move algorithm: src strings become empty
std::vector<std::string> dst2(3);
std::move(src.begin(), src.end(), dst2.begin());
// dst2: {"a", "b", "c"}, src: {"", "", ""}
```

Q48. How does `std::generate` and `std::generate_n` work?**Answer:**

`std::generate` calls a nullary callable (no arguments) for each element in a range and assigns the result. `std::generate_n` does the same for `n` elements. Useful for filling containers with sequences, random numbers, or any computed pattern.

Example:

```
#include <random>

std::vector<int> v(10);

// Generate sequence 0..9
int n = 0;
std::generate(v.begin(), v.end(), [&n]() { return n++; });

// Generate random numbers
std::mt19937 rng(42);
std::uniform_int_distribution<int> dist(1, 100);
std::generate(v.begin(), v.end(),
    [&]() { return dist(rng); });

// generate_n with output iterator
std::generate_n(std::back_inserter(v), 5,
    []() { return 99; });
```

Q49. What is `std::unique` and how is it different from `std::unique_ptr`?**Answer:**

`std::unique` (algorithm) removes consecutive duplicates in a sorted range, moving unique elements to the front and returning an iterator to the new end — must be followed by `erase` to actually remove. `std::unique_ptr` is a smart pointer for exclusive ownership. Completely different tools, similar name.

Example:

```
std::vector<int> v = {1, 1, 2, 2, 2, 3, 4, 4, 5};

// std::unique: removes consecutive duplicates
```

```

auto newEnd = std::unique(v.begin(), v.end());
// v = {1,2,3,4,5, ?,?,?,?} – logical end at newEnd

// Must erase to actually shrink
v.erase(newEnd, v.end());
// v = {1,2,3,4,5}

// NOTE: must be sorted first for all-duplicates removal!
std::vector<int> unsorted = {3,1,3,2,1};
std::sort(unsorted.begin(), unsorted.end());
unsorted.erase(std::unique(unsorted.begin(), unsorted.end()),
               unsorted.end());
// {1,2,3}

```

Q50. What are `std::inner_product`, `std::partial_sum`, and `std::adjacent_difference`?

▶ Answer:

All three are in `<numeric>`. `inner_product` computes the dot product of two ranges (sum of pairwise products), with optional custom ops. `partial_sum` computes prefix sums (each element = sum of all preceding). `adjacent_difference` computes differences between consecutive elements (inverse of `partial_sum` for arithmetic sequences).

📄 Example:

```

#include <numeric>

std::vector<int> a={1,2,3}, b={4,5,6};

// Dot product: 1*4+2*5+3*6 = 32
int dot = std::inner_product(a.begin(), a.end(), b.begin(), 0);
std::cout << dot; // 32

// Prefix sums
std::vector<int> v={1,2,3,4,5};
std::partial_sum(v.begin(), v.end(), v.begin());
// v: {1,3,6,10,15}

// Adjacent differences
std::vector<int> d;
std::adjacent_difference(v.begin(), v.end(),
                        std::back_inserter(d));
// d: {1,2,3,4,5} (recovers original)

```

SECTION 11: Strings, Searching & Numeric

Q51. How do `std::count` and `std::count_if` work?

▶ Answer:

`std::count` counts how many elements equal a given value. `std::count_if` counts elements satisfying a predicate. Both are $O(n)$. For sorted containers prefer `lower_bound`/`upper_bound` for $O(\log n)$ count, or the container's own `count()` member.

📄 Example:

```
std::vector<int> v = {1,2,3,2,1,2,4,2};

std::cout << std::count(v.begin(), v.end(), 2); // 4

std::cout << std::count_if(v.begin(), v.end(),
    [](int x){ return x % 2 == 0; }); // 5

// For sorted containers, prefer member count or bounds
std::multiset<int> ms(v.begin(), v.end());
std::cout << ms.count(2); // 4 - O(log n + count)
```

Q52. What is `std::search` and `std::search_n`?

▶ Answer:

`std::search` finds the first occurrence of a subsequence within a range. `std::search_n` finds *n* consecutive occurrences of a value. In C++17, `std::search` accepts a `Searcher` object (e.g. Boyer-Moore) for sub-linear string searching.

📄 Example:

```
std::string text = "the quick brown fox jumps";
std::string pat = "brown";

// Basic search: O(n*m)
auto it = std::search(text.begin(), text.end(),
    pat.begin(), pat.end());
if (it != text.end())
    std::cout << std::distance(text.begin(), it); // 10

// C++17 Boyer-Moore for large texts
auto it2 = std::search(text.begin(), text.end(),
    std::boyer_moore_searcher(pat.begin(), pat.end()));
```

Q53. How does `std::minmax_element` work and what does it return?

▶ Answer:

`std::minmax_element` finds both the minimum and maximum in a range in a single pass using at most $3n/2$ comparisons (more efficient than two separate calls). Returns a pair of iterators: `{min_iterator, max_iterator}`.

📄 Example:

```
std::vector<int> v = {3,1,4,1,5,9,2,6,5};

auto [minIt, maxIt] = std::minmax_element(v.begin(), v.end());

std::cout << *minIt; // 1
std::cout << *maxIt; // 9
std::cout << std::distance(v.begin(), minIt); // index 1

// Compare: two passes
auto mi = std::min_element(v.begin(), v.end()); // 1 pass
auto ma = std::max_element(v.begin(), v.end()); // 1 pass
// minmax_element: ~1.5x passes total - slightly faster
```

Q54. What is `std::rotate` and how is it used?

▶ Answer:

`std::rotate` performs an in-place left rotation of a range. The element at `new_first` becomes the new first element of the range. It returns an iterator to the position where the original first element now resides. Runs in $O(n)$. Useful for implementing cyclic shifts, or bringing an element to the front.

Example:

```
std::vector<int> v = {1,2,3,4,5,6,7};

// Rotate so 4 is at front
auto it = std::rotate(v.begin(), v.begin()+3, v.end());
// v: {4,5,6,7,1,2,3}
// it points to the old first element (1)
std::cout << *it; // 1

// Bring min element to front
auto minIt = std::min_element(v.begin(), v.end());
std::rotate(v.begin(), minIt, v.end());
```

Q55. How do `std::set_union`, `std::set_intersection`, and `std::set_difference` work?**▶ Answer:**

These algorithms operate on sorted ranges (not just sets) and produce sorted outputs. `set_union` merges two sorted ranges into one containing elements from either. `set_intersection` yields elements in both. `set_difference` yields elements in the first but not the second. All run in $O(n+m)$ time.

Example:

```
std::vector<int> a={1,2,3,4,5}, b={3,4,5,6,7};
std::vector<int> out;

std::set_union(a.begin(),a.end(),b.begin(),b.end(),
    std::back_inserter(out));
// out: {1,2,3,4,5,6,7}

out.clear();
std::set_intersection(a.begin(),a.end(),b.begin(),b.end(),
    std::back_inserter(out));
// out: {3,4,5}

out.clear();
std::set_difference(a.begin(),a.end(),b.begin(),b.end(),
    std::back_inserter(out));
// out: {1,2}
```

SECTION 12: Advanced & Expert-Level Questions

Q56. What is perfect forwarding in the context of STL `emplace` methods?**▶ Answer:**

Perfect forwarding preserves the value category (lvalue/rvalue) of arguments when forwarding to another function via `std::forward<Args>`. `Emplace` methods use variadic templates and `std::forward` to pass

arguments directly to element constructors without copying. This is why `emplace_back(args...)` can construct in-place.

Example:

```
template<typename... Args>
void myEmplace(std::vector<std::string>& v, Args&&... args) {
    v.emplace_back(std::forward<Args>(args)...);
}

// All of these are handled perfectly:
std::vector<std::string> v;
myEmplace(v, "hello");           // const char* -> string ctor
std::string s = "world";
myEmplace(v, s);                 // lvalue: copy
myEmplace(v, std::move(s));      // rvalue: move
myEmplace(v, 5, '!');            // fill ctor: "!!!!!"
```

Q57. What is `std::inplace_merge` and when is it preferred?

Answer:

`std::inplace_merge` merges two consecutive sorted sub-ranges `[first, middle)` and `[middle, last)` in-place. With extra memory: $O(n)$; without: $O(n \log n)$. It is used as a building block for merge sort on containers where you can't use additional buffers, or when you need to merge two sorted halves of a vector.

Example:

```
std::vector<int> v = {1,3,5,7, 2,4,6,8};
//                sorted   sorted

auto mid = v.begin() + 4;
std::inplace_merge(v.begin(), mid, v.end());
// v: {1,2,3,4,5,6,7,8}

// Use case: merge-sort implementation
void mergeSort(auto first, auto last) {
    if (last - first <= 1) return;
    auto mid = first + (last-first)/2;
    mergeSort(first, mid);
    mergeSort(mid, last);
    std::inplace_merge(first, mid, last);
}
```

Q58. Explain `std::partition`, `std::stable_partition`, and their use in sorting.

Answer:

`std::partition` rearranges elements so those satisfying a predicate come first. It is NOT stable (relative order of equal elements not preserved) and runs in $O(n)$. `std::stable_partition` maintains relative order within each group: $O(n \log n)$ without extra memory, $O(n)$ with it. These are the core operation of quicksort's partition step.

Example:

```
std::vector<int> v = {3,1,4,1,5,9,2,6,5};

// Partition: evens first, then odds
auto pivot = std::partition(v.begin(), v.end(),
    [](int x){ return x%2==0; });
// [even elements | odd elements]
// pivot points to first odd element
```

```
// stable_partition: preserves relative order
std::stable_partition(v.begin(), v.end(),
    [](int x){ return x < 5; });
// [1,1,2,4,3 | 5,9,6,5] – order preserved within groups
```

Q59. What is `std::next_permutation` and how does it work?

▶ Answer:

`std::next_permutation` rearranges elements to the next lexicographically greater permutation and returns true. Returns false and sorts to the smallest permutation when already at maximum. Iterating with `next_permutation` over all permutations of a sorted range yields them in lexicographic order, useful for combinatorial algorithms.

Example:

```
std::vector<int> v = {1,2,3};

// Enumerate all 6 permutations
do {
    for (int x : v) std::cout << x << " ";
    std::cout << "\n";
} while (std::next_permutation(v.begin(), v.end()));
// 1 2 3
// 1 3 2
// 2 1 3
// 2 3 1
// 3 1 2
// 3 2 1
```

Q60. What is `std::sample` (C++17) and how does it differ from `std::shuffle`?

▶ Answer:

`std::sample` selects `n` elements randomly from a range without replacement, using reservoir sampling. `std::shuffle` randomly permutes all elements in a range. `sample` works on forward iterators and is $O(n)$; `shuffle` requires random access iterators and is $O(n)$. Use `sample` for random subsets; `shuffle` for random ordering.

Example:

```
#include <random>
#include <algorithm>

std::vector<int> v(100);
std::iota(v.begin(), v.end(), 1); // 1..100

std::mt19937 rng(std::random_device{}());

// Sample 5 random elements (no replacement)
std::vector<int> sample(5);
std::sample(v.begin(), v.end(), sample.begin(), 5, rng);

// Shuffle all elements
std::shuffle(v.begin(), v.end(), rng);
```

Q61. What are transparent comparators and heterogeneous lookup?

▶ Answer:

Transparent comparators (`std::less<void>`, or any type with `is_transparent`) enable lookup in ordered containers using types different from the key without constructing a key object. In C++14+, `std::map<std::string, V, std::less<>>` lets you call `find()` with `string_view` or `const char*` without allocating a `std::string`.

Example:

```
// Without transparent comparator: allocates string!
std::map<std::string, int> m1;
m1["hello"] = 1;
m1.find("hello"); // const char* -> constructs std::string

// With transparent comparator: no allocation
std::map<std::string, int, std::less<>> m2;
m2["hello"] = 1;
std::string_view sv = "hello";
m2.find(sv); // Uses string_view directly
m2.find("hello"); // Uses const char* directly
// No std::string temporary created!
```

Q62. What is the "Sequence of Sequences" pattern and `std::vector<std::vector<T>>>`?

Answer:

`std::vector<vector<T>>` stores rows as separate heap allocations, causing poor cache locality and many allocations. For performance-critical code, prefer a single flat vector with manual index arithmetic, or use a 2D matrix class. The flat layout is often 5-10x faster due to cache effects.

Example:

```
// Poor: many heap allocations
std::vector<std::vector<int>> matrix2d(rows,
    std::vector<int>(cols, 0));

// Better: single allocation, manual indexing
std::vector<int> flat(rows * cols, 0);
auto at = [&](int r, int c) -> int& {
    return flat[r * cols + c];
};
at(2, 3) = 42;

// Or use std::mdspan (C++23) for multi-dim views
// std::mdspan<int, std::extents<int, R, C>> view(flat.data());
```

Q63. How does node extraction (C++17) work in `std::map` and `std::set`?

Answer:

C++17 adds `extract()` which removes a node from a container and returns it as a `node_handle` without any allocation or copying. This allows cheap transfer of elements between containers of the same type, changing of keys in a map (previously impossible without `erase+insert`), and moving non-copyable elements.

Example:

```
std::map<int, std::string> m1 = {{1, "one"}, {2, "two"}};
std::map<int, std::string> m2;

// Extract node – no allocation, no copy
auto node = m1.extract(1);
// m1: {2: "two"}, node holds {1, "one"}

// Modify the key (impossible without extract before!)
node.key() = 10;
```

```
// Insert into another map
m2.insert(std::move(node));
// m2: {10:"one"}
```

Q64. What is `std::merge` vs `std::inplace_merge`?

► Answer:

`std::merge` takes two separate sorted ranges and writes the merged result to a third range — requires $O(n+m)$ output space. `std::inplace_merge` merges two adjacent sorted sub-ranges in place — no separate output needed but may use extra memory internally. Use `merge` when you have separate containers; `inplace_merge` for merging halves.

▣ Example:

```
std::vector<int> a = {1,3,5,7};
std::vector<int> b = {2,4,6,8};
std::vector<int> out;

// std::merge: two separate ranges -> output
std::merge(a.begin(), a.end(),
           b.begin(), b.end(),
           std::back_inserter(out));
// out: {1,2,3,4,5,6,7,8}

// inplace_merge: adjacent sorted halves
std::vector<int> v = {1,3,5,7, 2,4,6,8};
std::inplace_merge(v.begin(), v.begin()+4, v.end());
// v: {1,2,3,4,5,6,7,8}
```

Q65. What are `std::views::iota`, `std::views::repeat`, and `std::views::zip` (C++23)?

► Answer:

`iota(start)` or `iota(start, end)` generates a lazy sequence of incrementing values — no storage needed. `repeat(v, n)` produces n copies of a value lazily. `zip(r1, r2, ...)` lazily pairs elements from multiple ranges into tuples. All are range adaptors that compose with other views.

▣ Example:

```
#include <ranges>

// iota: generate 0..9 lazily
for (int i : std::views::iota(0, 10))
    std::cout << i << " ";

// zip (C++23): pair two ranges
std::vector<int> ids = {1,2,3};
std::vector<std::string> names = {"a","b","c"};
for (auto [id, name] : std::views::zip(ids, names))
    std::cout << id << ":" << name << " ";
// 1:a 2:b 3:c

// Composable
auto sq_odds = std::views::iota(1, 20)
    | std::views::filter([](int x){ return x%2!=0; })
    | std::views::transform([](int x){ return x*x; });
```

Q66. What is copy elision and guaranteed RVO in C++ and how does it affect STL usage?**▶ Answer:**

Return Value Optimization (RVO) and Named RVO (NRVO) construct the return value directly in the caller's storage, eliding copy/move. C++17 mandates RVO for certain cases (prvalues), making returning STL containers by value zero-cost. This means returning `std::vector<T>` is always efficient in C++17+.

📄 Example:

```
std::vector<int> makeVector() {
    std::vector<int> v(1000000);
    std::iota(v.begin(), v.end(), 0);
    return v; // Guaranteed RVO in C++17
}

// C++17: guaranteed zero copies
auto v = makeVector(); // Constructed in place – no move!

// PRE-C++17: might copy/move (but NRVO usually applies)
// DO NOT write: return std::move(v); // Defeats RVO!
```

Q67. What is the difference between `std::sort` and `std::ranges::sort`?**▶ Answer:**

`std::ranges::sort` (C++20) is a constrained version using concepts: it requires `std::sortable` range (random access iterators, swappable, comparable). It takes a range instead of iterator pairs, supports projections (transform elements before comparing), and has cleaner error messages via concepts. The underlying algorithm is the same.

📄 Example:

```
#include <algorithm>
#include <ranges>

std::vector<std::string> words={"banana","apple","cherry"};

// Classic sort by full string
std::ranges::sort(words);

// Ranges sort with PROJECTION (sort by length!)
std::ranges::sort(words, {}, &std::string::size);
// ["apple","banana","cherry"] (by length: 5,6,6)

// Custom comparator + projection
std::ranges::sort(words,
    std::greater{}, // descending
    &std::string::size); // by length
```

Q68. How does `std::priority_queue` support custom comparators for complex objects?**▶ Answer:**

Provide a Comparator type as the third template parameter. The comparator must implement strict weak ordering where `comp(a,b)==true` means a should come BEFORE b in the queue (a has lower priority). This is the opposite of intuition — for a max-heap, `comp` returns `a < b` (smaller items are "less prioritized").

📄 Example:

```
struct Task {
    int priority;
    std::string name;
```

```
};

struct TaskCmp {
    bool operator()(const Task& a, const Task& b) const {
        return a.priority < b.priority; // min first
    }
};

std::priority_queue<Task, std::vector<Task>, TaskCmp> pq;
pq.push({3, "Low"});
pq.push({1, "Critical"});
pq.push({2, "Medium"});
std::cout << pq.top().name; // Critical (priority=1)
```

Q69. What are `std::iota` and `std::fill` and how do they differ?

▶ Answer:

`std::fill` assigns a constant value to every element in a range. `std::iota` assigns sequentially increasing values starting from an init value, incrementing with `++`. Both are $O(n)$. Use `fill` to initialize all elements to the same value; `iota` for sequences, indices, or counters.

📄 Example:

```
#include <numeric>

std::vector<int> v(10);

// fill: all elements = 7
std::fill(v.begin(), v.end(), 7);
// v: {7,7,7,7,7,7,7,7,7,7}

// iota: 0,1,2,...
std::iota(v.begin(), v.end(), 0);
// v: {0,1,2,3,4,5,6,7,8,9}

// iota from 100
std::iota(v.begin(), v.end(), 100);
// v: {100,101,102,...,109}
```

Q70. What are the performance implications of `std::list::splice`?

▶ Answer:

`std::list::splice` moves elements from one list to another (or within the same list) in $O(1)$ time — just pointer relinking, no copy or allocation. It is the one operation where list definitively beats vector (vector can't do $O(1)$ arbitrary moves). Useful for implementing LRU caches, intrusive data structures, or task queues.

📄 Example:

```
std::list<int> a = {1,2,3,4,5};
std::list<int> b = {10,20,30};

// Move all of b into a before element 3
auto it = std::find(a.begin(), a.end(), 3);
a.splice(it, b); // O(1)! No copies
// a: {1,2,10,20,30,3,4,5} b: {}

// Move single element within same list – O(1)
a.splice(a.begin(), a, std::prev(a.end()));
// Moves last element to front
```

Q71. What is the difference between `std::vector<bool>` and other `std::vector` specializations?**Answer:**

`std::vector<bool>` is a controversial specialization that packs bits — 8 booleans per byte instead of 1 bool per byte. This saves memory but breaks the promise that elements have addresses: `operator[]` returns a proxy object, not `bool&`. You cannot take the address of an element. Prefer `std::vector<char>` or `std::deque<bool>` if you need reference semantics, or `std::bitset` for fixed-size bit arrays.

Example:

```
std::vector<bool> vb = {true, false, true};

// DOES NOT COMPILE: no reference to bit
// bool& ref = vb[0]; // Error!

// Returns proxy, not bool&
auto proxy = vb[0]; // proxy object

// Safe usage:
bool val = vb[0]; // OK, copies value

// Alternative: avoid the specialization
std::vector<char> vc = {1, 0, 1};
char& ref = vc[0]; // OK, real reference
```

Q72. What are the guarantees of `std::list::sort` vs `std::sort`?**Answer:**

`std::list` has its own `sort()` member because `std::sort` requires random access iterators which `list` does not provide. `std::list::sort` uses merge sort, runs in $O(n \log n)$, and is stable. It operates by relinking nodes without copying elements. Never call `std::sort` on a list — use the member function.

Example:

```
std::list<int> lst = {5, 2, 8, 1, 9, 3};

// std::sort(lst.begin(), lst.end()); // ERROR!
// std::sort requires random access iterators

// Correct: use member sort
lst.sort(); // O(n log n), stable, merge sort
// lst: {1,2,3,5,8,9}

// Custom comparator
lst.sort(std::greater<int>());
// lst: {9,8,5,3,2,1}
```

Q73. How do you implement an LRU cache using STL containers?**Answer:**

An LRU cache combines `std::list` (for $O(1)$ move-to-front) with `std::unordered_map<key, list::iterator>` (for $O(1)$ lookup). On access: find in map, splice node to front. On insert: add to front; if over capacity, erase from map and pop from back. This gives $O(1)$ amortized get and put.

Example:

```

template<typename K, typename V>
class LRUCache {
    int cap;
    std::list<std::pair<K,V>> lst;
    std::unordered_map<K,
        typename std::list<std::pair<K,V>>::iterator> mp;
public:
    LRUCache(int c) : cap(c) {}
    V get(const K& k) {
        auto it = mp.find(k);
        if (it == mp.end()) return V{};
        lst.splice(lst.begin(), lst, it->second);
        return it->second->second;
    }
    void put(const K& k, const V& v) {
        if (mp.count(k)) lst.erase(mp[k]);
        lst.push_front({k, v});
        mp[k] = lst.begin();
        if ((int)mp.size() > cap) {
            mp.erase(lst.back().first);
            lst.pop_back();
        }
    }
};

```

Q74. What is `std::tie` and how is it used for lexicographic comparison?

► Answer:

`std::tie` creates a tuple of lvalue references. It enables elegant lexicographic comparisons of structs by creating tuples of their fields and comparing them — tuples compare lexicographically by default. It also enables tuple unpacking (assigning multiple return values).

Example:

```

struct Person {
    std::string name;
    int age;
    bool operator<(const Person& o) const {
        // Compare by name then age, one line!
        return std::tie(name, age) < std::tie(o.name, o.age);
    }
};

// Tuple unpacking
int a, b, c;
std::tie(a, b, c) = std::make_tuple(1, 2, 3);

// Ignore fields with std::ignore
std::tie(a, std::ignore, c) = std::make_tuple(10, 99, 30);
// a=10, b unchanged, c=30

```

Q75. What are the differences between `std::copy_backward` and `std::move_backward`?

► Answer:

`std::copy_backward` copies elements from `[first,last)` to result, starting from `last-1` going backward. This is essential when source and destination overlap and destination is to the right — forward copy would corrupt data. `std::move_backward` does the same but moves rather than copies. The result iterator points to one past the end of the destination.

 Example:

```
std::vector<int> v = {1,2,3,4,5,6,7};

// Shift elements right by 2 (overlapping ranges)
// WRONG: std::copy(v.begin(), v.begin()+5, v.begin()+2)
// CORRECT: copy_backward for right-shift
std::copy_backward(v.begin(), v.begin()+5, v.end());
// v: {1,2,1,2,3,4,5}

// move_backward for non-copyable types
std::vector<std::unique_ptr<int>> up(5);
// ... fill up ...
// std::move_backward(up.begin(), up.begin()+3, up.end());
```

Q76. How does `std::unordered_map` handle hash collisions? **Answer:**

`std::unordered_map` uses separate chaining: each bucket is a linked list of elements with the same hash. On lookup: compute hash, go to bucket, linear-scan the chain. Performance degrades to $O(n)$ if all elements collide (same bucket). Good hash functions minimize collisions. You can call `rehash()` to increase bucket count and reduce collisions.

 Example:

```
// Deliberately bad hash – all keys map to same bucket
struct BadHash {
    size_t operator()(int) const { return 42; } // Always 42!
};

std::unordered_map<int,int,BadHash> bad;
// Insertions still work but find() is  $O(n)$ 

// Good practice: inspect and resize
std::unordered_map<int,int> um;
um.reserve(10000); // Pre-size
um.max_load_factor(0.4); // Stay sparse
std::cout << um.bucket_count(); // ~25000
```

Q77. What is the difference between `std::all_of`, `std::any_of`, and `std::none_of`? **Answer:**

All three test predicates over a range and short-circuit. `all_of` returns true if predicate is true for every element. `any_of` returns true if predicate is true for at least one element. `none_of` returns true if predicate is false for all elements. All return true for empty ranges (vacuous truth) except `any_of` which returns false.

 Example:

```
std::vector<int> v = {2, 4, 6, 8, 10};

auto isEven = [](int x){ return x%2==0; };

std::cout << std::all_of(v.begin(),v.end(),isEven); // 1
std::cout << std::any_of(v.begin(),v.end(),
    [](int x){ return x>8; }); // 1 (10>8)
std::cout << std::none_of(v.begin(),v.end(),
    [](int x){ return x<0; }); // 1

// Empty range: all_of->true, any_of->>false, none_of->true
std::vector<int> empty;
```

```
std::cout << std::all_of(empty.begin(), empty.end(), isEven); // 1
```

Q78. What is `std::make_heap`, `std::push_heap`, and `std::pop_heap`?

▶ Answer:

These algorithms operate on ranges as heaps. `make_heap` converts an arbitrary range into a max-heap in $O(n)$. `push_heap` adds a newly appended element to the heap in $O(\log n)$ — you must `push_back` first. `pop_heap` moves the max to the end of the range in $O(\log n)$ — you must then `pop_back` to remove it. Used to build custom priority queues.

▮ Example:

```
std::vector<int> v = {5,3,8,1,9,2};

// Build heap: O(n)
std::make_heap(v.begin(), v.end());
// v[0] is max (9)

// Add element
v.push_back(15);
std::push_heap(v.begin(), v.end()); // O(log n)
// v[0] is now 15

// Remove max
std::pop_heap(v.begin(), v.end()); // O(log n)
// Max (15) moved to v.back()
v.pop_back();

// Heap sort!
std::sort_heap(v.begin(), v.end()); // O(n log n)
```

Q79. What is `std::span` and how does it enable safe array access?

▶ Answer:

`std::span<T,N>` (C++20) is a non-owning, bounds-tracked view over a contiguous sequence. The extent `N` can be dynamic (`std::dynamic_extent`) or static (known at compile time). It eliminates pointer+size passing, prevents buffer overruns via `at()`, and works with any contiguous container. It is the preferred way to write functions that accept any contiguous range.

▮ Example:

```
#include <span>

void printSpan(std::span<const int> s) {
    for (int x : s) std::cout << x << " ";
    std::cout << "\n";
    std::cout << "size: " << s.size() << "\n";
}

int arr[] = {1,2,3,4,5};
std::vector<int> v = {6,7,8};

printSpan(arr);           // Works with C array
printSpan(v);             // Works with vector
printSpan({arr, 3});      // First 3 elements only

// Subspan
std::span<int> s(arr, 5);
auto sub = s.subspan(1, 3); // elements 1,2,3
```

Q80. How do you efficiently find and remove duplicates from a vector of complex objects?**▶ Answer:**

Sort the vector using a custom comparator, then apply the erase-remove idiom with `std::unique` using a custom equality predicate. For large data sets or when order must be preserved, use an `unordered_set` to track seen elements and partition/erase based on it.

Example:

```
struct Person { std::string name; int age; };

std::vector<Person> v = {"Alice", 30}, {"Bob", 25},
                      {"Alice", 30}, {"Carol", 28}};

// Sort by name+age, then erase duplicates
std::sort(v.begin(), v.end(), [](auto& a, auto& b){
    return std::tie(a.name, a.age) < std::tie(b.name, b.age);
});
v.erase(std::unique(v.begin(), v.end(), [](auto& a, auto& b){
    return a.name==b.name && a.age==b.age;
}), v.end());
// v has 3 unique Persons
```

Q81. What is the Ranges pipeline and what problem does it solve over traditional algorithms?**▶ Answer:**

Traditional STL algorithms are not composable — each returns a container and requires an explicit pass. Ranges pipelines are lazy and composable: you chain views without intermediate allocations. The pipe operator `|` chains views, evaluation is deferred until iteration, and no intermediate containers are created.

Example:

```
#include <ranges>

std::vector<int> data(1000000);
std::iota(data.begin(), data.end(), 1);

// Traditional: two passes, two temp vectors!
std::vector<int> tmp, result;
std::copy_if(data.begin(), data.end(),
    std::back_inserter(tmp), [](int x){return x%3==0;});
std::transform(tmp.begin(), tmp.end(),
    std::back_inserter(result), [](int x){return x*x;});

// Ranges: single lazy pass, zero allocations
auto rng = data
    | std::views::filter([](int x){ return x%3==0; })
    | std::views::transform([](int x){ return x*x; });
for (int x : rng) { /* use x */ }
```

Q82. What is `std::chrono` and how does it work with STL?**▶ Answer:**

`std::chrono` provides a type-safe time library with durations (milliseconds, seconds), time points, and clocks (`system_clock`, `steady_clock`, `high_resolution_clock`). It integrates with STL containers to create timed queues, and with mutexes for timed locks. `steady_clock` is best for measuring elapsed time as it does not go backwards.

Example:

```
#include <chrono>
using namespace std::chrono;

auto start = steady_clock::now();

// Benchmark STL sort
std::vector<int> v(1000000);
std::iota(v.begin(), v.end(), 0);
std::shuffle(v.begin(), v.end(), std::mt19937{42});
std::sort(v.begin(), v.end());

auto end = steady_clock::now();
auto ms = duration_cast<milliseconds>(end-start).count();
std::cout << "Sort took: " << ms << " ms\n";
```

Q83. What is a Trie and how would you implement one using STL containers?

Answer:

A Trie (prefix tree) is a tree for strings where each node represents a character. Using STL, each node can be represented with an `unordered_map<char, Node*>` for children and a `bool` for end-of-word. `unique_ptr` avoids manual memory management. Supports $O(L)$ insert, search, and prefix-search where L is string length.

Example:

```
struct TrieNode {
    std::unordered_map<char,
        std::unique_ptr<TrieNode>> children;
    bool isEnd = false;
};

class Trie {
    TrieNode root;
public:
    void insert(const std::string& word) {
        TrieNode* cur = &root;
        for (char c : word) {
            if (!cur->children.count(c))
                cur->children[c]=std::make_unique<TrieNode>();
            cur = cur->children[c].get();
        }
        cur->isEnd = true;
    }
    bool search(const std::string& word) {
        TrieNode* cur = &root;
        for (char c : word) {
            if (!cur->children.count(c)) return false;
            cur = cur->children[c].get();
        }
        return cur->isEnd;
    }
};
```

Q84. What is `std::reference_wrapper` and when is it needed?**▶ Answer:**

`std::reference_wrapper<T>` (from `std::ref/std::cref`) wraps a reference in a copyable, assignable object. STL containers require copyable/movable elements — you cannot store plain references in a vector. `reference_wrapper` solves this, enabling containers of references. Also used with `std::bind` and thread constructors to pass by reference.

📄 Example:

```
#include <functional>

int a=1, b=2, c=3;

// Cannot do: std::vector<int&> v; // Error!

// reference_wrapper: copyable reference
std::vector<std::reference_wrapper<int>> v = {a, b, c};

for (int& x : v) x *= 10; // Modifies originals!
std::cout << a; // 10

// Sort a vector by the referenced values
std::sort(v.begin(), v.end(),
    [](int a, int b){ return a < b; });
```

Q85. How does `std::mismatch` find the first difference between two ranges?**▶ Answer:**

`std::mismatch` compares two ranges element by element and returns a pair of iterators pointing to the first position where they differ. If ranges are equal, returns `{end1, end2}`. Accepts an optional custom comparator. Useful for string comparison, finding divergence points, or testing sorted order.

📄 Example:

```
std::string s1 = "Hello World";
std::string s2 = "Hello Earth";

auto [it1, it2] = std::mismatch(s1.begin(), s1.end(),
                               s2.begin());
std::cout << std::distance(s1.begin(), it1); // 6
std::cout << *it1; // W
std::cout << *it2; // E

// Compare with custom comparator (case-insensitive)
auto [i1, i2] = std::mismatch(
    s1.begin(), s1.end(), s2.begin(),
    [](char a, char b){ return tolower(a)==tolower(b); });
```

Q86. What is the CRTP (Curiously Recurring Template Pattern) in STL context?**▶ Answer:**

CRTP is a pattern where a class inherits from a template instantiated with itself. It enables static polymorphism (no virtual dispatch overhead) and is used for mixins. In STL context, it appears in custom iterator implementations (`std::iterator_facade` style), operator generation patterns, and policy-based design.

📄 Example:

```
// CRTP for adding equality ops from less-than
template<typename Derived>
```

```

struct Comparable {
    friend bool operator>(const Derived& a, const Derived& b)
    { return b < a; }
    friend bool operator<=(const Derived& a, const Derived& b)
    { return !(a > b); }
    friend bool operator>=(const Derived& a, const Derived& b)
    { return !(a < b); }
};

struct Point : Comparable<Point> {
    int x, y;
    bool operator<(const Point& o) const {
        return std::tie(x,y) < std::tie(o.x,o.y);
    }
};
// Point gets >, <=, >= for free – usable with STL algorithms

```

Q87. What is concept-constrained programming in C++20 and how does it improve STL usage?

► Answer:

C++20 Concepts allow you to express requirements on template parameters. STL algorithms in C++20 use concepts (e.g. `std::sortable`, `std::input_range`, `std::predicate`) for cleaner error messages and enabling function overloading based on type properties. You can write your own concepts to constrain STL-based generic functions.

📄 Example:

```

#include <concepts>

// Custom concept: Summable range
template<typename R>
concept SummableRange = std::ranges::input_range<R> &&
    std::integral<std::ranges::range_value_t<R>>;

// Function constrained by concept
template<SummableRange R>
auto sumRange(const R& r) {
    return std::reduce(r.begin(), r.end(), 0LL);
}

std::vector<int> v = {1,2,3,4,5};
std::cout << sumRange(v); // 15
// sumRange("string") -> CLEAR error: not SummableRange

```

Q88. How do you use `std::back_inserter`, `std::front_inserter`, and `std::inserter`?

► Answer:

These create output iterator adaptors that insert into containers. `back_inserter` calls `push_back`, `front_inserter` calls `push_front` (for deque/list), and `inserter(cont, pos)` calls `insert` at a position. Use them as output iterators for algorithms like `copy`, `transform`, and `set_union`.

📄 Example:

```

std::vector<int> src = {1,2,3,4,5};
std::vector<int> dst;

// back_inserter: auto push_back
std::copy(src.begin(), src.end(), std::back_inserter(dst));

```

```
// front_inserter: auto push_front (deque/list only)
std::deque<int> dq;
std::copy(src.begin(), src.end(), std::front_inserter(dq));
// dq: {5,4,3,2,1} (reversed!)

// inserter: insert at specific position
std::set<int> s = {1,3,5};
std::copy(src.begin(), src.end(),
    std::inserter(s, s.begin())); // Inserts into set
// s: {1,2,3,4,5}
```

Q89. What is lazy evaluation in C++ and how do Ranges implement it?

► Answer:

Lazy evaluation defers computation until results are needed. Range views implement this via custom iterators whose operator++ applies the filter/transform on demand. No intermediate storage is created. This enables infinite ranges (iota without end), pipeline composition without copies, and early termination without wasted work.

📄 Example:

```
#include <ranges>

// Infinite lazy range
auto naturals = std::views::iota(1);

// First 5 even squares – computed lazily
auto result = naturals
    | std::views::filter([](int x){ return x%2==0; })
    | std::views::transform([](int x){ return x*x; })
    | std::views::take(5);

for (int x : result)
    std::cout << x << " "; // 4 16 36 64 100
// Only computed enough elements – no full traversal!
```

Q90. How do you write a thread-safe wrapper around an STL container?

► Answer:

STL containers are NOT thread-safe for concurrent reads+writes. Wrap with a mutex. Use RAII lock_guard/scoped_lock for exception safety. For read-heavy workloads, use shared_mutex with shared_lock for readers and unique_lock for writers (reader-writer lock). Atomic containers are rarely needed — wrapping is almost always the right approach.

📄 Example:

```
#include <mutex>
#include <shared_mutex>

template<typename T>
class ThreadSafeVector {
    std::vector<T> data;
    mutable std::shared_mutex mtx;
public:
    void push_back(const T& val) {
        std::unique_lock lock(mtx); // Exclusive write
        data.push_back(val);
    }
    T at(size_t i) const {
```

```

        std::shared_lock lock(mtx); // Shared read
        return data.at(i);
    }
    size_t size() const {
        std::shared_lock lock(mtx);
        return data.size();
    }
};

```

Q91. What is `std::atomic` and how does it relate to STL containers?

▶ Answer:

`std::atomic<T>` provides lock-free atomic operations on fundamental types. It does NOT make containers thread-safe — each individual element access is atomic but multi-step container operations (size check + push) are not. Useful for counters, flags, and indices. For containers, use mutexes rather than atomic elements.

📄 Example:

```

#include <atomic>
#include <thread>

std::atomic<int> counter{0};
std::vector<std::thread> threads;

for (int i = 0; i < 10; ++i)
    threads.emplace_back([&]() {
        for (int j = 0; j < 1000; ++j)
            counter.fetch_add(1, std::memory_order_relaxed);
    });

for (auto& t : threads) t.join();
std::cout << counter; // Always 10000, no data race

```

Q92. What is the policy-based design and how does it manifest in STL allocators?

▶ Answer:

Policy-based design uses template parameters (policies) to swap algorithms or behaviors. STL uses this for allocators (how memory is managed) and comparators (how elements are ordered). C++17 polymorphic memory resources (`std::pmr`) extend this further with runtime-swappable allocation policies, enabling arena, pool, or stack allocation without changing container code.

📄 Example:

```

#include <memory_resource>

// Different allocation strategies for same container type
std::pmr::vector<int> heapVec; // Default: heap

std::byte stack[1024];
std::pmr::monotonic_buffer_resource arena(stack, 1024);
std::pmr::vector<int> stackVec(&arena); // Stack-backed

std::pmr::unsynchronized_pool_resource pool;
std::pmr::vector<std::pmr::string> poolVec(&pool); // Pool

// All are std::pmr::vector<int> – same type!
// Behavior controlled by memory resource at construction

```


Q93. How do `std::experimental::scope_exit` and RAI relate to STL containers?**Answer:**

RAII (Resource Acquisition Is Initialization) is a core C++ idiom where resource cleanup is tied to object lifetime. All STL containers are RAII — destructors clean up memory. Scope guards (like the proposed `scope_exit`) extend this to arbitrary cleanup actions. Writing RAII wrappers around STL operations ensures exception safety.

Example:

```
// Simple scope guard for STL cleanup
template<typename F>
struct ScopeGuard {
    F f;
    ~ScopeGuard() { f(); }
};

void process(std::vector<int>& v) {
    int original_size = v.size();
    ScopeGuard rollback{[&]() {
        v.resize(original_size); // Always restore on exit
    }};
    v.push_back(100);
    v.push_back(200);
    // If exception thrown, rollback fires
    if (v.size() > 1000) throw std::overflow_error{"too big"};
    rollback.f = []{}; // Disarm if success needed
}
```

Q94. What is `std::experimental::fixed_capacity_vector` (or the Boost equivalent)?**Answer:**

A `fixed_capacity_vector` has static capacity (no heap allocation) but variable size up to that capacity. Ideal for embedded systems, real-time code, or any context requiring deterministic allocation. It stores elements in aligned stack storage. Boost provides `boost::container::static_vector`; C++26 may add `std::inplace_vector`.

Example:

```
// Using Boost (or C++26 std::inplace_vector)
#include <boost/container/static_vector.hpp>

// Max 8 elements, zero heap allocation
boost::container::static_vector<int, 8> sv;
sv.push_back(1);
sv.push_back(2);
// sv.capacity() == 8, sv.size() == 2

// Works with STL algorithms
std::sort(sv.begin(), sv.end());

// push_back beyond capacity throws or UB (impl-defined)
// Great for: interrupt handlers, game loops, RT code
```

Q95. How does `std::multiset` differ from `std::priority_queue` for sorted access?**Answer:**

Both maintain elements in sorted order. `multiset` stores all elements and supports iteration, erase by value or iterator, and lookup — it is a full sorted container. `priority_queue` only exposes the max element, has no iteration, and does not support erasing arbitrary elements. Use `multiset` when you need flexible access; `priority_queue` for pure priority-queue semantics.

Example:

```
// Multiset: full sorted access
std::multiset<int> ms = {3,1,4,1,5,9,2,6};
for (int x : ms) std::cout << x << " "; // 1 1 2 3 4 5 6 9
ms.erase(ms.find(1)); // Erase only first 1

// priority_queue: only top() accessible
std::priority_queue<int> pq;
for (int x : {3,1,4,1,5,9,2,6}) pq.push(x);
std::cout << pq.top(); // 9
// Cannot iterate or erase arbitrary element

// Performance: pq O(log n) push/pop, no overhead for order
// ms: O(log n) + node allocation per element, slower
```

Q96. What is concept `std::invocable` and how do you use it to constrain STL-like functions?

Answer:

`std::invocable<F, Args...>` checks that `F` is callable with `Args...`. `std::regular_invocable` adds the semantic constraint that the call does not modify observable state. Use these in `requires` clauses or template constraints to create readable, well-constrained generic algorithms.

Example:

```
#include <concepts>

// Constrain a custom algorithm
template<
    std::ranges::input_range R,
    std::invocable<std::ranges::range_value_t<R>> F
>
void myForEach(R&& range, F&& func) {
    for (auto&& elem : range)
        std::invoke(func, elem);
}

std::vector<int> v = {1,2,3};
myForEach(v, [](int x){ std::cout << x; }); // OK
// myForEach(v, 42); // Compile error: int not invocable
```

Q97. What is `std::experimental::generator` (C++23) and how does it integrate with Ranges?

Answer:

`std::generator<T>` (C++23) is a coroutine-based lazy range generator. Using `co_yield`, you write imperative code that produces values on demand. The generator satisfies the range concept, so it integrates seamlessly with Ranges pipelines — you can apply `views::filter`, `transform`, `take`, etc. directly to coroutine generators.

Example:

```
// C++23
#include <generator>

std::generator<int> fibonacci() {
    int a = 0, b = 1;
    while (true) {
```

```

        co_yield a;
        auto c = a + b;
        a = b; b = c;
    }
}

// Compose with Ranges views
auto fibs = fibonacci()
    | std::views::take(10)
    | std::views::filter([](int x){ return x%2==0; });

for (int x : fibs) std::cout << x << " "; // 0 2 8 34

```

Q98. How do you detect and avoid the ABA problem when using STL with lock-free structures?

▶ Answer:

The ABA problem occurs in lock-free code when a value changes from A to B and back to A, causing a CAS to succeed incorrectly. Relevant when implementing lock-free stacks or queues outside of STL. Solutions include hazard pointers, epoch-based reclamation, or tagged pointers. STL does not provide lock-free containers — you must use external libraries.

🔧 Example:

```

// Illustration of ABA risk in a lock-free pop
// Thread 1: reads head=A, reads next=B
// Thread 2: pops A, pops B, pushes A back
// Thread 1: CAS(head, A, B) succeeds – but B is freed!

// Mitigation: tagged pointers (lower bits as version)
struct Tagged {
    void* ptr;
    uintptr_t tag; // increment on each CAS
};
// Use std::atomic<Tagged> with compare_exchange

// Best practice: use std::mutex for correctness,
// or a proven library like Folly or TBB for lock-free.

```

Q99. What is the SSO threshold and how does it vary across implementations?

▶ Answer:

The Small String Optimization (SSO) threshold — the maximum length string stored inline without heap allocation — varies by implementation: libstdc++ uses 15 chars (16 with null terminator on 64-bit), libc++ uses 22 chars, MSVC uses 15 chars. Strings at or below the threshold have no heap allocation. You can probe this experimentally by checking if `data()` is inside `sizeof(string)` bytes of `&s`.

🔧 Example:

```

auto isSSO = [](const std::string& s) {
    const char* data = s.data();
    const char* obj = reinterpret_cast<const char*>(&s);
    return data >= obj && data < obj + sizeof(s);
};

// Test threshold experimentally
for (int len = 10; len <= 30; ++len) {
    std::string s(len, 'x');
    std::cout << len << ": "

```

```

        << (isSSO(s) ? "SSO" : "heap") << "\n";
    }
    // libstdc++: SSO up to len=15, heap from 16
    // libc++:   SSO up to len=22, heap from 23

```

Q100. What are the key differences between C++11, C++14, C++17, and C++20 STL additions?

► Answer:

C++11 added unordered containers, smart pointers, move semantics, emplace methods, initializer_list. C++14 added heterogeneous lookup (transparent comparators), make_unique. C++17 added std::optional, variant, any, string_view, parallel algorithms, node handles (extract), std::filesystem, pmr. C++20 added Ranges, span, coroutines, concepts, atomic_ref, jthread, semaphore, latch, barrier, format.

📄 Example:

```

// C++11
std::unordered_map<int,int> um;
auto up = std::unique_ptr<int>(new int(5));

// C++14
auto up14 = std::make_unique<int>(5); // make_unique
std::set<std::string, std::less<>> s14; // Transparent

// C++17
std::optional<int> opt = 42;
std::string_view sv = "hello";
std::sort(std::execution::par, v.begin(), v.end());

// C++20
auto view = v | std::views::filter([](int x){return x>0;});
std::span<int> sp(v);
std::cout << std::format("Value: {}\n", 42);

```